

Lesson 2 Stock Deliver



This lesson is customized for users who have bought the deluxe version.

1. Program Logic

Taking advantage of advanced technology and smart devices, intelligent warehousing system can efficiently manipulate the process of stock enter, stock deliver, sorting, packaging, distribution, etc. Compared with human warehousing, intelligent warehousing can update the information in time, including inventory, logistics status, etc., to ensure that the company can master the real warehousing data and adjust the inventory.

In this lesson, we are going to learn how the ArmPi FPV perform “stock deliver” function.

First, we need to set the location of the goods on each shelf layer, then set the external input to decide which layer to pick up and the picking order.

After receiving the data, deliver subroutine will control the body of robotic arm to turn to the corresponding shelf. After the picking angle and height is adjusted by deliver subroutine, the robotic arm will approach and pick up the goods. Next, control the robotic arm to deliver and place the goods.

The source code is located in `/home/armpi_fpv/src/warehouse/scripts/out.py`.

```
24 # Warehousing
25 # If not declared, the length and distance units used are all m
26
27 # Initialization
28 __target_data = ()
29 __isRunning = False
30 org_image_sub_ed = False
31 lock = threading.RLock()
32 ik = ik_transform.ArmIK()
33
34 # Initialization position
35 def initMove():
36     with lock:
37         bus_servo_control.set_servos(joints_pub, 1500, ((1, 75
38             ), (2, 500), (3, 80), (4, 825), (5, 625), (6, 500)))
39         rospy.sleep(2)
40
41 # reset
42 def reset():
43     global __target_data
44     __target_data = ()
45
46 # app initializtion
47 def init():
48     rospy.logininfo("out Init")
```

2. Operation Step

i The entered command must be strictly distinguished between uppercase and lowercase and spaces, and the keywords support the "TAB" key completing.

Let's take the example of making the robotic arm deliver R1, R2 and R3 goods on the left shelf in turns.

2.1 Preparation

- ① Please place the map on even desk and the robotic arm and shelf should be placed on the corresponding area on the map.
- ② Place 3 blocks on each layer of the right shelf respectively. You can place them randomly on each layer, but ensure that one block is placed on one layer and on the center of the shelf.
- ③ Turn on the robotic arm and wait to finish.

2.2 Enter the Game

- ① Turn on ArmPi FPV and connect to Raspberry Pi desktop through NoMachine.
- ② Click “Applications” button and select “Terminal Emulator” in pop-up interface to open the command line terminal.



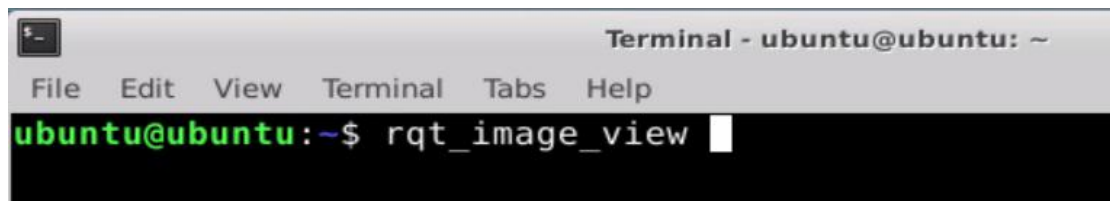
- ③ Enter “**rosservice call /out/enter "{}"**” command, then press “Enter” to start the game. After the game starts, a print prompt will appear, as shown in the figure below.

```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ rosservice call /out/enter "{}"  
success: True  
message: "enter"  
ubuntu@ubuntu:~$
```

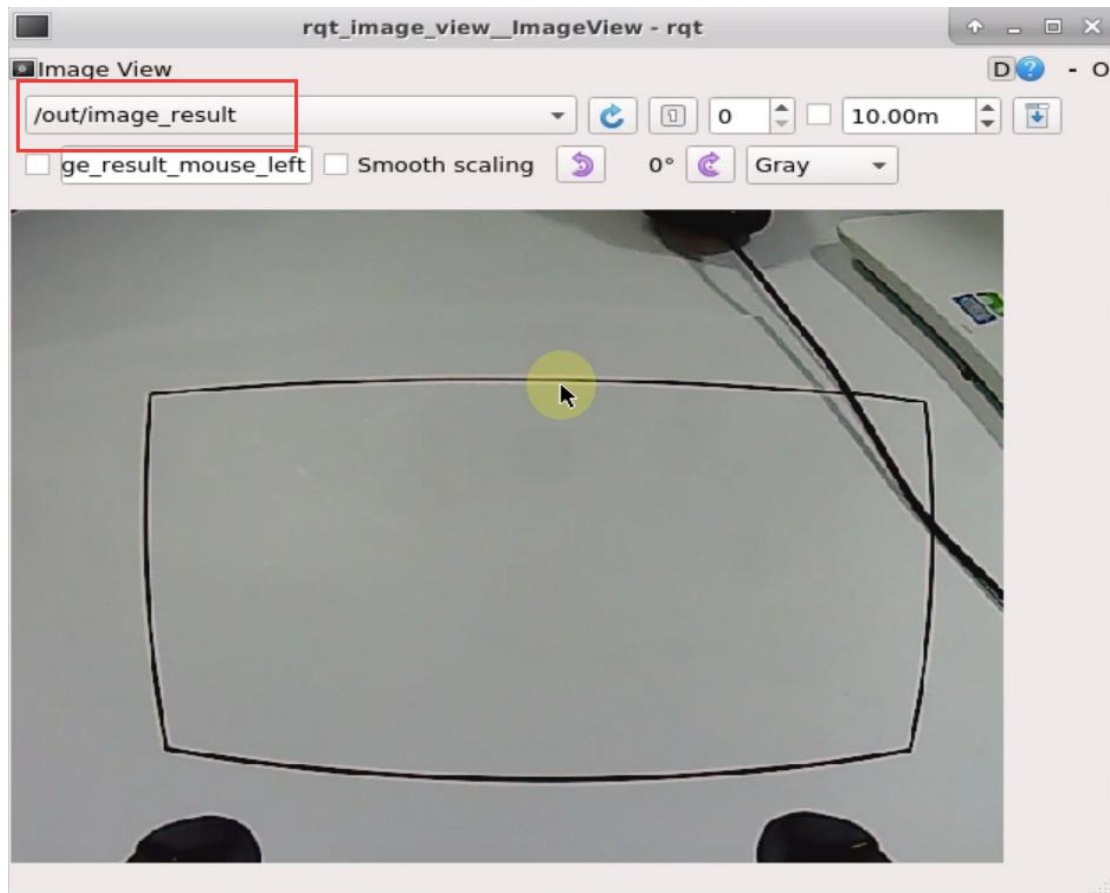
2.3 Turn On the Image

① When entering the game, we need to turn on image returned by camera with rqt. Please go back to the desktop and click the “Application” icon at bottom-left corner to open a new terminal.

② Enter “**rqt_image_view**” command and press “Enter”. Then open rqt after few seconds.



③ As shown in the following figure, select “/out/image_result” and other settings remain the same.



Note: after the image is turned on, please be sure to select the topic corresponding to

the game you play. Otherwise, the recognition process can not be displayed normally if you start other games.

2.4 Start the Game

① Please go back to the terminal opened in “2.2 Enter the Game” and enter **rosservice call /in/set_running "data: true"** command. When the prompt, shown in the red box, appears, it means you start the game successfully.

```
ubuntu@ubuntu:~$ rosservice call /out/set_running "data: true"
success: True
message: "set_running"
ubuntu@ubuntu:~$
```

② When the game starts, we need to set the parameter, including target goods and the deliver order. The total number of the layers of the two shelves is 6, hence the robotic arm can deliver 6 items once the program is executed.

Through setting the parameter, we can designate the goods to be delivered. The delivering order is consistent with the parameter we input. Let's take the example of making the robotic arm deliver R1, R2 and R3 goods on the left shelf in turns. And enter the following command.

```
rosservice call /out/set_target "position:
```

```
- 'R1'
```

```
- 'R2'
```

```
- R3'"
```

```
ubuntu@ubuntu:~$ rosservice call /out/set_target "position:
- 'R1'
> - 'R2'
> - 'R3'"
```

③ Sets the type of input parameter in the parameter command, and you can refer to the table below for their meanings and built-in parameters.

Type	Meanings	Built-in parameter
position	Number of shelf	Left Shelf (from bottom to top): L1, L2, L3 Right Shelf (from bottom to top): R1, R2, R3

④ Pay attention to the following points when entering parameters.

(1) If you are entering multiple parameters (as shown in the figure above), you must wrap between each parameters. And please strictly follow this **format, dash + space + 'shelf number'**.

(2) In order to avoid errors caused by input method, it is recommended to use the Tab key completing, and enter the parameters in the corresponding position.

(3) If you enter the parameters after Tab key completing, then you need to delete the content after the first parameter before wrapping. After all the parameters are entered, add double quotation marks after the last parameter. The instruction is as follow.

```
ubuntu@ubuntu:~$ rosservice call /out/set_target "position:
- ' ' " 1
```

```
ubuntu@ubuntu:~$ rosservice call /out/set_target "position:
- 'R1'
> - 'R2'
> - 'R3' 2
```

```
ubuntu@ubuntu:~$ rosservice call /out/set_target "position:
- 'R1'
> - 'R2'
> - 'R3' " 3
```

2.5 Pause and Quit the Game

You can enter “**rosservice call /in/set_running "data: false"”** command to pause the game.

```
ubuntu@ubuntu:~$ rosservice call /out/set_running "data: false"
success: True
message: "set_running"
ubuntu@ubuntu:~$
```

Referring to “2.4 Start the game”, you can change and add shelves and layers of shelves.

You can enter “**rosservice call /in/exit "{}”** command to quit the game.

```
ubuntu@ubuntu:~$ rosservice call /out/exit "{}"
success: True
message: "exit"
ubuntu@ubuntu:~$
```

Note: the current game will continue as Raspberry Pi is powered on. To avoid occupying too much RAM, please quit the current game according to the instruction above before playing other AI vision game.

If you want to turn off the image returned by camera, you can turn back to open rqt terminal and press “Ctrl+C”.

3. Function Realization

When the game starts, the robotic arm will deliver the goods on the corresponding shelf in turns in accordance with the number of shelf inputted in Step 2.4.

After delivered, the goods will be placed in a straight line in the recognition area. And the placing order depends on the location of the shelf where items were originally placed. If the items on each shelf are delivered, the placement positions of the items on recognition area, from left to right, are R1, R2, R3, L1, L2, L3.